AI Code Review — Murajea

Professional Code Analysis Report

File: UserService.cs

Date: 2026-03-26 02:56:03

EXECUTIVE SUMMARY

TOTAL ISSUES	CRITICAL	WARNINGS	FUNCTIONS
8	2	4	3

CODE QUALITY SCORE: 40%

DETAILED ANALYSIS

CreateUser

Line 9

Code:

```
public User CreateUser(string name, string email)
        {
            // مفيش :مشكلة validation للـ email
            // مفيش :مشكلة null check للـ name
            var user = new User
            {
                Name = name,
                Email = email,
                CreatedAt = DateTime.Now
            };

            _users.Add(user);
            return user;
        }
```

⚠️ Issues Found:

🔴 Critical: The function does not validate the email address before creating a new user

⚠️ Warning: The function does not check for null or empty name before creating a new user

ℹ️ Info: The function uses DateTime.Now which may not be suitable for all environments, consider using a more robust datetime solution

💡 Suggestion: Consider adding input validation and error handling to ensure the function creates valid users and handles potential exceptions

---

📌 GetUserById

Line 24

📝 **Code:**

```
public User GetUserById(int id)
        {
            // من غير ما تتحقق null مشكلة: ممكن ترجع
            return _users.FirstOrDefault(u => u.Id == id);
        }
```

⚠️ Issues Found:

⚠️ Warning: The function may return null without being checked, potentially causing a NullReferenceException

💡 Suggestion: Consider adding a null check or using the nullable reference type feature to handle the possibility of a null return value, and provide a meaningful default value or throw a custom exception if the user is not found

---

📌 UpdateUserEmailAsync

Line 30

📝 **Code:**

```
public async Task<bool> UpdateUserEmailAsync(int id, string newEmail)
        {
            // مشكلة: async method من غير CancellationToken
            var user = GetUserById(id);
            if (user == null) return false;

            user.Email = newEmail;
            await Task.Delay(100);
            return true;
        }
```

⚠️ Issues Found:

🔴 Critical: The async method UpdateUserEmailAsync is missing a CancellationToken parameter, which can lead to issues with cancellation and disposal of resources.

⚠️ Warning: The method GetUserById is called synchronously, which can block the calling thread and negate the benefits of using an async method.

⚠️ Warning: The method UpdateUserEmailAsync does not handle any potential exceptions that may occur during execution, which can lead to unexpected behavior and errors.

ℹ️ Info: The method UpdateUserEmailAsync uses a Task.Delay, which may not be the best approach for simulating a delay in an async method.

💡 Suggestion: Consider refactoring the method to include a CancellationToken parameter, use async-friendly methods for retrieving the user, and add proper exception handling to ensure the method is robust and reliable.